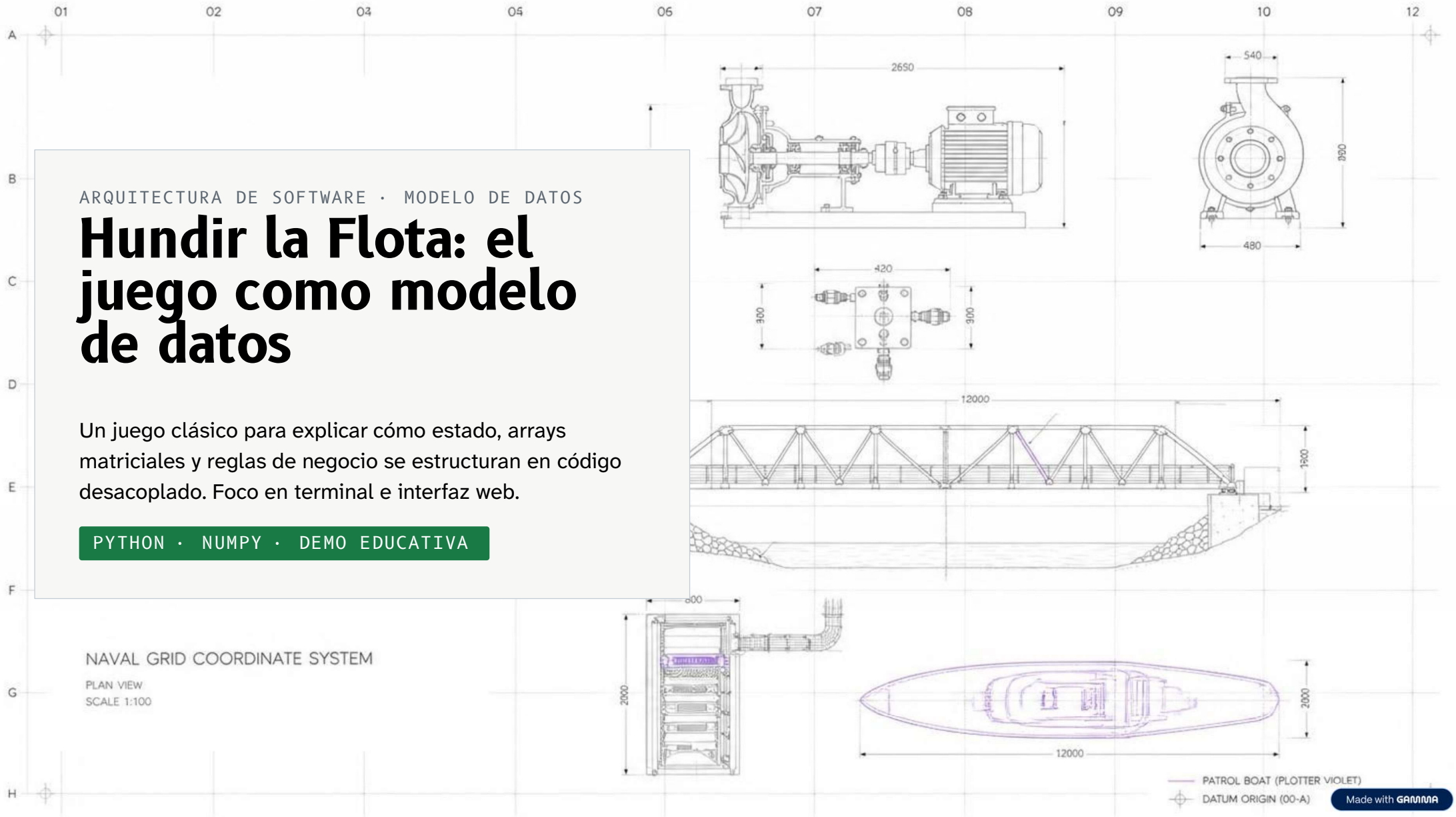
ARQUITECTURA DE SOFTWARE · MODELO DE DATOS

Hundir la Flota: el juego como modelo de datos

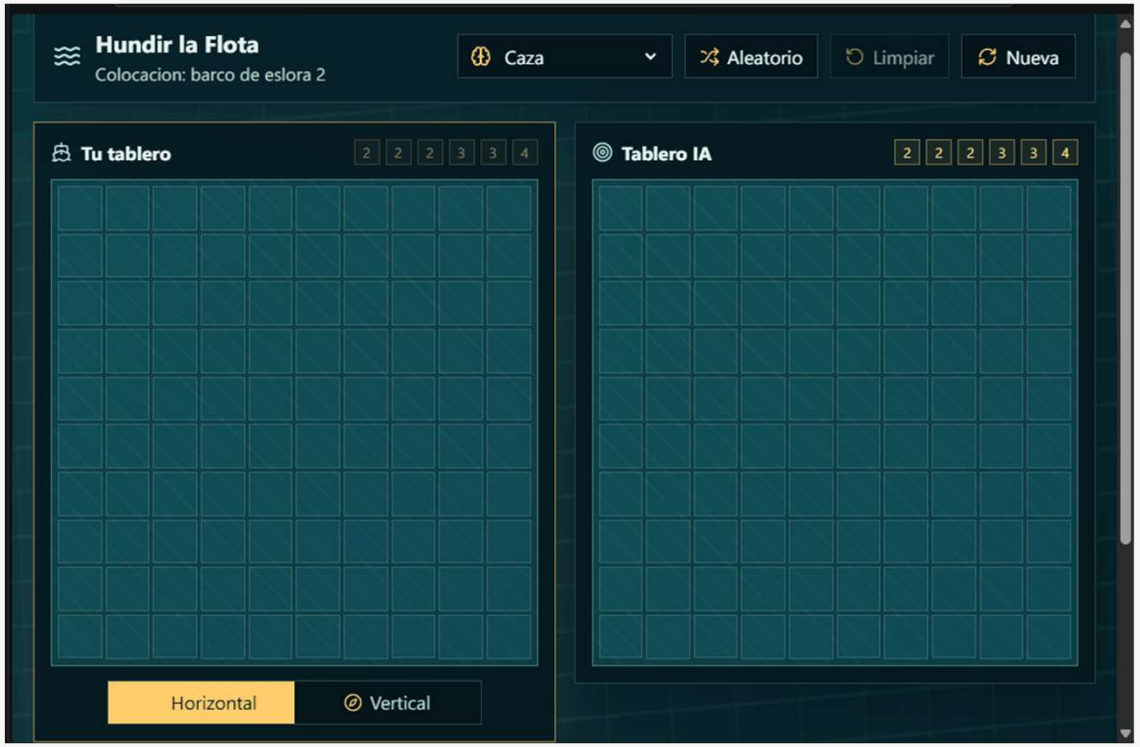
Un juego clásico para explicar cómo estado, arrays matriciales y reglas de negocio se estructuran en código desacoplado. Foco en terminal e interfaz web.

PYTHON · NUMPY · DEMO EDUCATIVA



Made with GAMMA





MODALIDAD

Aleatorio - Caza - Aprendizaje

TECNOLOGIAS

Python, Numpy, FastAPI, React, Vite

ALCANCE

Terminal y web

ENTORNO

Ejecución 100% local

Made with GAMMA

ESTRUCTURA ATÓMICA

Cada casilla tiene un estado y cada barco una longitud

El motor entiende el carácter; SIMBOLOS le asigna glifo y color para la terminal.

ESTADOS POSIBLES

`_` agua sin disparar · `O` barco · `A` fallo · `X` tocado · `H` hundido

COMPOSICIÓN DE LA FLOTA

Flota fija de seis barcos: `[2, 2, 2, 3, 3, 4]`

```
AGUA_VACIA = "_"
BARCO = "O"
TOCADO = "X"
AGUA = "A"
HUNDIDO = "H"

FLOTA = [2, 2, 2, 3, 3, 4]
```


Made with GAMMA

REPRESENTACIÓN BIDIMENSIONAL

NumPy: una matriz para el tablero

El tablero se modela como un array 2D de NumPy. Esto permite indexación directa por coordenadas fila/columna y vectorización en operaciones de filtrado.

INDEXACIÓN DIRECTA

Acceso O(1) mediante `tablero[fila, columna]`

MÁSCARA DE NIEBLA

Copia del array ocultando los barcos rivales sin alterar el estado real

```
def crear_tablero(tamano=10):
    return np.full((tamano, tamano), AGUA_VACIA)

def ocultar_barcos(tablero):
    visible = tablero.copy()
    visible[visible == BARCO] = AGUA_VACIA
    return visible
```


Made with GAMMA

ALGORITMO DE DISPOSICIÓN

La colocación elige ubicaciones legales sin solapar barcos

```
def colocar_barcos(tablero, esloras=FLOTA):
    tamano = tablero.shape[0]
    flota = []
    for eslora in esloras:
        candidatas = _candidatas(eslora, tamano)
        random.shuffle(candidatas)
        for barco in candidatas:
            if _cabe(barco, tablero):
                colocar_barco(barco, tablero)
                casillas = set(barco)
                flota.append({"vivas":
set(casillas), "todas": casillas})
                break
    return flota
```

-N: Guardar por esloras – Verificar configuración del tablero

1. ENUMERAR

`_candidatas` calcula todas las posiciones continuas horizontales y verticales que caben.

2. BARAJAR Y VALIDAR

`_cabe` comprueba que todas las casillas estén vacías; `colocar_barcos` baraja y usa la primera que cabe.

3. CONSTRUIR ÍNDICE

`preparar_bando` repite hasta colocar la flota completa y construye el índice casilla a barco; cada barco mantiene sus casillas `vivas` y `todas`.

Regla de contacto: los barcos no comparten casilla, pero sí pueden posicionarse en celdas contiguas.

Made with GAMMA

RESOLUCIÓN DE IMPACTOS

disparar convierte una casilla en resultado y actualiza el barco

```
def disparar(casilla, tablero, indice):
    estado = tablero[casilla]
    if estado in (TOCADO, AGUA, HUNDIDO):
        return "repetido"
    if estado == AGUA_VACIA:
        tablero[casilla] = AGUA
        return "agua"
    barco = indice[casilla]
    barco["vivas"].discard(casilla)
    if barco["vivas"]:
        tablero[casilla] = TOCADO
        return "tocado"
    for celda in barco["todas"]:
        tablero[celda] = HUNDIDO
    return "hundido"
```

01

Validar estado

Si la casilla ya recibió un disparo previo, retorna de inmediato `repetido` sin gastar turno.

02

Marcar agua

Si la celda estaba vacía, muta el valor a `AGUA` y retorna el resultado correspondiente.

03

Tocado o Hundido

Elimina la coordenada de `vivas`. Si el conjunto queda vacío, transforma todas sus celdas en `HUNDIDO`.

Made with GAMMA

INTELIGENCIA ARTIFICIAL

Tres estrategias: cada dificultad elige el disparo de otra forma

```
# Caza: tras un TOCADO, añade vecinos como objetivos
def elegir_disparo(self):
    while self.objetivos:
        casilla = self.objetivos.pop()
        if casilla in self.posicion:
            return casilla
    return self._al_azar()

# Aprendizaje: cada ventana válida suma peso a sus
celdas
peso = cantidad / len(validas)
for ventana in validas:
    for fila, col in ventana:
        puntuacion[fila][col] += peso
```

NIVEL 1 · ALEATORIA

Disparo al azar

Elige celdas libres mediante distribución uniforme; recuerda disparos realizados.

NIVEL 2 · MODO CAZA

Búsqueda por proximidad

Al acertar, explora los 4 vecinos ortogonales y alinea disparos a lo largo del eje.

NIVEL 3 · HEURÍSTICA

Mapa de densidad

Simula todas las ubicaciones posibles de los barcos restantes y dispara a la máxima probabilidad.

Made with GAMMA

FLUJO DE EJECUCIÓN

Un turno en terminal: main.py coordina y utils decide

```
def turno_maquina(estrategia, tablero, indice, activo):
    casilla = estrategia.elegir_disparo()
    resultado = utils.disparar(casilla, tablero, indice)
    barco = indice[casilla]["todas"] if resultado ==
"hundido" else None
    estrategia.registrar(casilla, resultado, barco)
    return resultado
```

1. **Inicialización:** `play.py` ejecuta `main()`; se selecciona nivel (1-3) y tamaño (por defecto 10x10, mín 6).
2. **Turno usuario:** `pedir_disparo()` valida sintaxis y `disparar()` actualiza la matriz enemiga.
3. **Turno máquina:** la IA calcula objetivo, ejecuta disparo y registra el resultado para refinar su memoria.
4. **Verificación final:** `quedan_barcos()` audita las flotas vivas y decide la victoria o el siguiente ciclo.

Made with GAMMA

DISEÑO ARQUITECTURAL

Las reglas están separadas del bucle de terminal y de la capa web

La lógica del tablero, la colocación y los disparos reside en un núcleo puro en Python. Tanto la consola como la interfaz web son clientes periféricos intercambiables.

```python
def _clone_fleet(flota: list[dict[str, set[tuple[int, int]]]]) -> list[dict[str, set[tuple[int, int]]]]:
    return [
        {"vivas": set(barco["vivas"]), "todas": set(barco["todas"])}
        for barco in flota
    ]
```

```python
def _board_payload(tablero: np.ndarray) -> list[list[str]]:
    return tablero.tolist()
```

CLIENTE TERMINAL

CLI Interactiva

`play.py`, `main.py` y `colores.py` manejan teclado, pausas y formato ANSI.

NÚCLEO COMPARTIDO

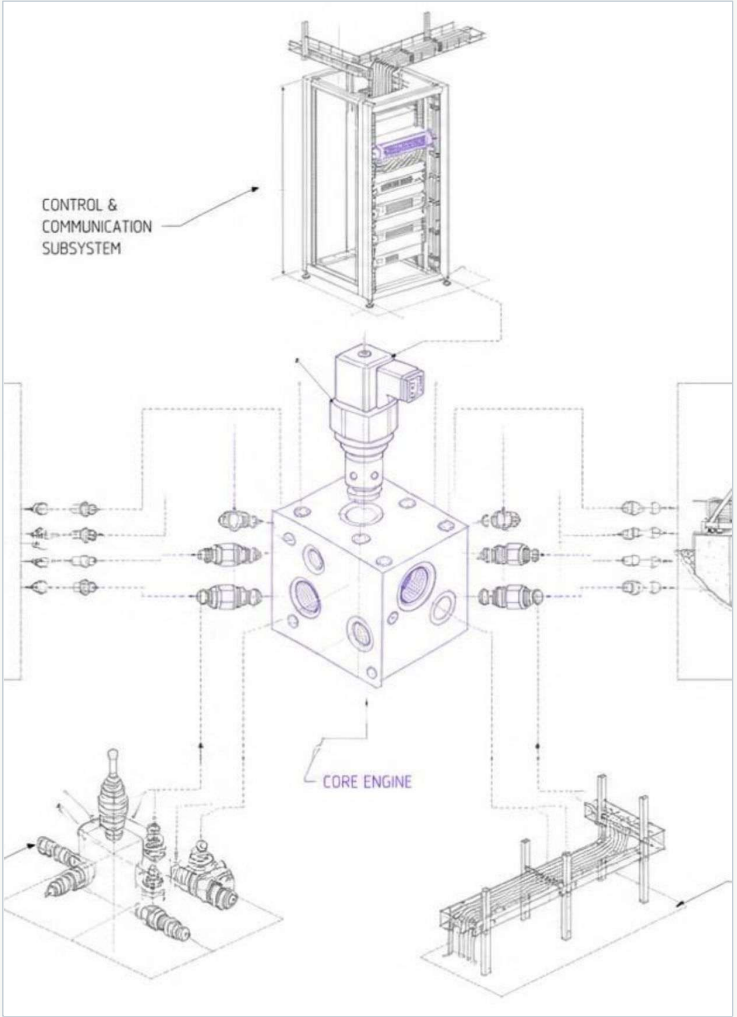
Lógica de Dominio

`utils.py` (matrices NumPy, flota, reglas de impacto) y estrategias de IA desacopladas.

CLIENTE WEB

Servicio y React

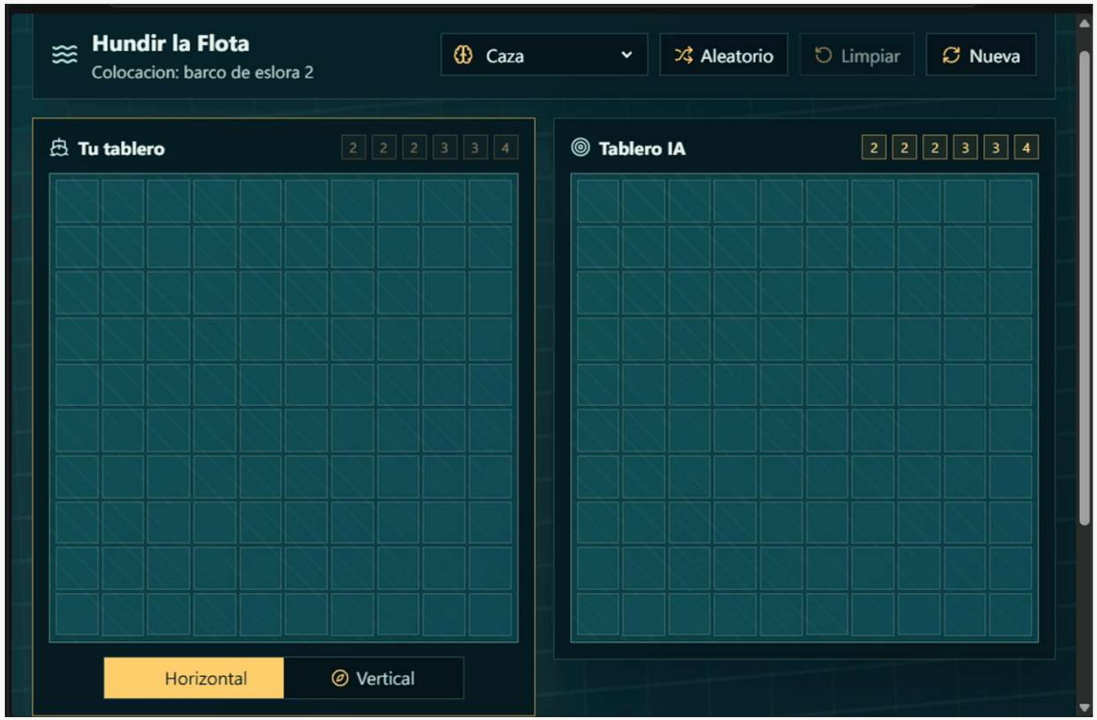
FastAPI expone endpoints HTTP; `GameSession` serializa matrices a JSON para React.



Made with GAMMA

EXTENSIÓN DE LA ARQUITECTURA

Web: reglas reutilizadas, interacción nueva



FLUJO

React, FastAPI, GameSession, utils e IA.

GAMESESSION

Añade fases, historial y serialización de la matriz NumPy a JSON.

INTERFAZ

Permite colocar barcos con clic, cambiar dificultad y ver el código de la última decisión de IA.

Made with GAMMA